

Using Unification-Based Techniques for More Accurate Equation Matching in Math Q&A Systems

Abstract

The diverse nature of mathematics makes it uniquely relevant to a wide variety of professional fields and facets of daily life. Math is applied in manufacturing cars and designing phones, and it is used in cooking recipes and calculating property taxes. The immense relevance of mathematics combined with the ever-increasing reliance on digital platforms for problem-solving has created the demand for math Q&A platforms, such as Math Stack Exchange, which are popular due to the ease at which they can be accessed from online. Unfortunately, one of the major drawbacks of these platforms is the fact that users are sometimes forced to wait for an unspecified amount of time before receiving an answer to their question. Math Q&A platforms mitigate this issue by recommending to the user similar questions that have already been answered. However, the algorithms that handle the recommendation process leave much room for improvement, as existing recommendation algorithms are capable of dealing with natural language, but less so math queries which frequently include equations that cannot be processed in the same way as natural language. This project aims to address this by researching the applicability of the idea found in *unification*, traditionally used to demonstrate equivalence of math information such as sets and expressions, can be applied to increase the accuracy of the answers that a math Q&A algorithm recommends. This research presents new ideas for the math Q&A community to consider, concluding that there is potential in the applicability of unification to improve the performance of math Q&A systems.

Introduction

Mathematics plays a critical role in a wide variety of disciplines, including engineering, economics, computer science, biology, etc. It is the foundation for modern advancements in medicine [1], transportation [2], entertainment [17], and many other fields. Outside of a professional setting, math is also an integral part of everyday life, appearing in ordinary everyday settings such as finance management, shopping, cooking, scheduling, etc. The relevance of mathematics extends far into daily life, yet many perceive mathematics as an inherently difficult topic [3], leading to reluctance to engage with it [4]. This is especially problematic as the demand for technical literacy increases [5]. But this affects individuals even outside of a career, as a lack of math skills increases the likelihood of making illogical and poor decisions [6].

The reluctance to engage with math does not reflect the importance of math, and highlights the need to make math learning more accessible. If learning math can be perceived as a less daunting task, then unwillingness to engage with it will diminish. Learning math can be made less intimidating by making math learning more accessible. In a world heavily reliant on the internet, one of the simplest and most accessible ways to learn is through using technology, and therefore online math Q&A systems have become an answer to the problem over the past

couple decades as a way to conveniently get answers to math questions [7]. Math Q&A systems allow users to ask questions as a user and answer existing questions as an answer provider. Some Math Q&A systems have a very large user base of people that are knowledgeable about math who are willing to play the role of answer provider as a hobby, all the way to people who professionally practice math at a high level. These systems provide a convenient way of connecting those who have math questions with those who are able to answer math questions, for a wide range of math topics and levels of difficulty.

The convenience of these math Q&A systems promotes math learning, but one big problem is that sometimes a user has to wait an unspecified amount of time to receive an answer, if it ever comes, perhaps defeating the convenience gained by being accessible from digital platforms. Often, a user's question is time-sensitive because they have a deadline to meet at work, or a due date to meet at school. And quite often a user simply does not want to wait for an unknown amount of time, as they might lose interest or become frustrated with the inability to find an answer in a timely manner. Math Q&A systems attempt to handle this issue by recommending to the question-asker existing similar questions that have already been answered, but this process is heavily reliant upon the quality of the recommendations. Poorly matched recommendations can discourage users, leaving them dissatisfied and potentially harming their desire to learn math. Improving the accuracy of these recommendations is thus a critical step towards enhancing the user experience on math Q&A platforms, and by extension a critical step towards improving math education.

To address this challenge, we explore the application of unification—a technique traditionally used in formal proofs to show the equivalence between hierarchical math structures like sets and expressions—to enhance the accuracy of math Q&A systems that recommend to the user relevant existing questions that have been answered. The proposed algorithm for matching math equations will compare math equations represented in MathML format and generate a score representing the similarity of the equations. The proposed model first arranges the MathML elements that make up the equations, such as operators and operands, into an array format that is easy to iterate over. Then a score is generated from the processed MathML elements based on the similarity of the elements, how many similar elements are found in a row, and the size of both equations.

There are a small handful of other instances of research that tackle the problem of math Q&A through equation-matching [4], but no previous research has taken the approach described above, and this research aims to assess its novelty and viability. This approach is also notable for its speed and simplicity, outperforming some of the most accurate modern generative AI systems, such as GPT-4.0.

Related Works

The recognition of room for improvement in the accuracy of math Q&A systems has driven research into bridging the accuracy gap of math information retrieval (IR) compared to

traditional, natural language IR. Researchers have recognized this problem and have devised methods for more effective consideration of formulas, equations, symbols, and other math terms that traditional IR systems struggle to handle [8]. This section provides a brief overview of a few notable systems addressing these challenges.

- MIaS (Math Indexer and Searcher) [4]: MIaS generates alternative versions of math equations by rearranging commutative expressions and generalizing constants, then indexes these variants by similarity, based on how drastic the changes performed are. Equations in the user's math question are matched against the indexed equations, determining which indexed equations are most similar to the user's equations, in order to identify relevant answers.
- EgoMath [10]: Similar to MIaS, EgoMath alters and indexes equations, but focuses on math content without accompanying semantic information, such as TEX-formatted math on Wikipedia. This enables searches on platforms where explicit semantic encoding is lacking, which makes it difficult for existing systems to interpret the meaning of math equations.
- Tangent-S [11]: Tangent-S uses symbol layout trees to represent equations as tree structures, explicitly capturing the relationship between operands and operators, as well as avoiding conversion errors. Considering this information improves similarity assessments by distinguishing structurally different but visually similar equations.
- Variable Typing [12]: Variable Typing links math symbols with their natural language context, creating a typed formula index. This method enhances the training and accuracy of math Q&A models compared to those without typed formula indexing.
- ARQMath and ARQMath-2 [13, 14]: These systems address math Q&A with both text and equations, retrieving relevant answers by evaluating the similarity between equations and the text surrounding them. These systems use data from Math Stack Exchange to evaluate their ability to retrieve accurate and relevant answers.

Our math Q&A approach to enhance the performance of retrieving relevant answers to math questions in math Q&A systems shares some similarities with some of the above models, such as the consideration of math equations represented in hierarchical MathML trees. Having said that, the proposed model introduces the novel combination of a depth first search, unification to match the math symbols in the equation, and a scoring system that considers both the depth score based on common consecutive elements and the complexity difference between the equations.

Methodology

Our approach to improving the accuracy of math Q&A systems by determining the similarity of math equations is accomplished in large part through the matching of like math operators and operands, which is a crucial component of unification. "Like" math operators and operands are operators and operands that can be matched with other similar operators and operands, such as constants matching with other constants, or variables matching with other variables. The more matched elements, the higher the score, and the more similar the algorithm

deems the two equations. The matching is done by using *unification*, an algorithmic process frequently used for theorem proofs and logic programming. Unification, in this context, is the determination of the solution for a system of two equations. It does so by comparing the operators and operands found in two equations in order to determine if one equation is similar to another equation. Operators and operands that can be matched with another operator or operand are said to be unifiable.

The designing of an algorithm that matches operators and operands, determining if one equation can be transformed into the other, could have an application in math Q&A systems, where the unification algorithm must determine the similarity of equations through comparison in order to retrieve the most relevant response to a user's query. In order to make unification applicable to math Q&A, we have designed a unification algorithm that is compatible with math equations specified in MathML. In MathML, math equations are represented in XML, using tags such as `<mo>` for operators, `<mi>` for variables, `<mn>` for literals (see Table (3) for a list of all MathML tags), where the relationships are represented in a hierarchical manner, suitable for an XML structured language (see Figure 1.0 and Figure 2.0, where equations are represented in standard math notation, and Figure 1.1 and Figure 2.1 depict their respective MathML representations).

Our math Q&A algorithm is made up of two parts: a traversal of the MathML tree representing the math equations being compared that yields two *arrays of arrays*, and a scoring that compares the two arrays of arrays. The two arrays of arrays represent the math equations being compared (see detailed discussion on arrays of arrays in the next section), and they are in array format for ease of comparison, as it is simpler to compare strings in an array than it is to compare elements of a MathML file parsed into Python *Elements* using Python's *ElementTree* module. The comparison determines how similar they are to each other, and therefore how similar the questions they represent are to each other. The final output is a number in the range 0 - 1, where 1 represents the two equations being identical, and 0 represents the two equations being completely dissimilar (for a glance of our overall unification process, see Figure 3.0 for the pseudocode of the proposed algorithm).

Traversal

Our math Q&A algorithm applies a depth-first search on the two equations being compared using the *ElementTree* module built into Python, generating two arrays of arrays. The arrays of arrays represent the branches of the equation in tree (XML) format. One array of arrays is derived from the equation in the user's query, referred to as the Right Tree, whereas another array of arrays is derived from the equation in the answer to be matched with the user's math question, referred to as the Left Tree. The subarrays represent the path to a specific leaf node, starting with the 'math' tag, which is the root of all MathML equations (see Figure 1.2 and Figure 2.2 for the array of arrays representation of the equation seen in Figure 1.0 and Figure 2.0).

Our math Q&A algorithm injects numbers into the arrays of arrays that represent the various branches of a tag. The purpose is to ensure that when matching is performed, the differences between the denominator and numerator, base and exponent, base and subscript, base and over-symbol, and base and under-symbol are accounted for. While the difference between the branches of the above tags needs to be accounted for when matching, the difference between what the tags `<mi>` (representing variables) and `<mo>` (representing constants) represent should not be considered. Instead `<mi>` tags should match with other `<mi>` tags, and the same for `<mo>` tags. For example, given that the query is $a+b$, then $c+d$ is likely a relevant answer, despite the fact that the variables represented by the `<mi>` tag do not match. Hence why the arrays representing the branches terminate with tags rather than what they actually represent, preventing the need to account for discrepancies in what those tags represent, as the discrepancies in what they represent are usually insignificant and distract from the task of determining similarity (see Figure 1.2 and Figure 2.2, which do not include what a tag actually represents, as seen in their respective MathML representation in Figure 1.1 and Figure 2.1 respectively).

Scoring

We compute the score that represents the degree to which the branches of any two equations match, and by extension, the degree to which the two equations being compared match. The score is calculated based on two sub-scores: depth score and complexity score.

Depth Score

Depth score is calculated by comparing each branch in the Right Tree to each branch in the Left Tree to determine which branch in the Left Tree is most similar to each branch in the Right Tree. The depth score calculation utilizes dynamic programming to find the longest contiguous subsequence of MathML tags found in both the Left Tree branch and Right Tree branch. In other words, the depth score for each branch is measured by determining the number of tags both branches have in common, in a row. The calculation of the depth score does not consider the MathML `<math>` tag found at the root of every branch, which serves the purpose of allowing MathML to embed directly in XML, to ensure a more accurate score.

For example, in Figure 2.2, branch/array 4 (of the Right Tree) would match with branch/array 5 (of the Left Tree) in Figure 1.2 with a score of 4/5. 4 is the number of common elements in both arrays in a row (`mrow`, `mfrac`, `0`, `mrow`), and 5 is the number of elements (`mrow`, `mfrac`, `0`, `mrow`, `mi`) in the Right Tree branch, excluding `'math'`. This process is repeated until each branch in the Right Tree has a depth score, at which time the algorithm has the information it needs to calculate the total depth score.

After the depth score is calculated, a weight is applied to increase the accuracy of the depth score to better reflect how similar the branches being compared are. As the algorithm determines the depth score for each branch, it also keeps track of weighted tags found in each branch. Weighted tags are tags that need special consideration when comparing branches because they frequently represent large structural differences in the equations. This means that a

weighted tag has a greater effect on the structure of an equation compared to a *unweighted* tag. The presence of a weighted tag in one branch but not the other means that the two branches have a large structural difference, and so the weights of the weighted tags are applied to the depth score, lowering the score based on how much the weighted tags affect the structure of that branch of the equation.

All weighted tags and their respective weights can be found in Table (2). The weighted tags `<mover>`, `<munder>`, and `<munderover>` are used for annotations like accents, limits, or combined notation, and have weights of 0.005. The weighted tags `<mfrac>`, `<msup>`, `<msubsup>`, and `<msqrt>` are used for annotations like fractions, superscript, superscript and subscript, and square roots, respectively and have weights of 0.5. The weight of 0.005 is assigned because those tags affect the structure of an equation more than any other tag. Since those tags alone can greatly affect the structure of an equation, the weight value is very close to 0, significantly lowering the depth score. The weight of 0.5 is assigned because the tags that have that weight affect the structure of the equation more than an unweighted tag, but not to the extent that the tags with a weight of 0.005 do. These tags alone can affect the structure of an equation significantly, but hardly as significant as a tag with a weight of 0.005.

For example, x^y and x/y would be deemed very similar based solely on the depth score of the unweighted branches, but in reality, the two equations are actually quite different. While the branch representing “x” would match the branch representing “x” in the other tree, and likewise with “y”, their places in the equations as a whole also affects the degree of similarity of the two equations as a whole. Therefore, the fact that one tree has an `<msup>` tag and the other an `<mfrac>` tag is considered. Since the branches do not have these tags in common, the depth score for these branches must reflect that, which is accomplished by the application of weights on top of the depth score, lowering it and therefore representing the difference that arises due to one branch containing a fraction and the other branch containing superscript. Without considering weights, the two branches in the equation x/y would both have a depth score of $3/5$, as the branch `[math, mrow, mfrac, 0, mrow, mi]` in the equation x/y would match with the branch `[math, mrow, msup, 0, mrow, mi]` in the equation x^y (math tag not included). This score is lowered to $3/20$ with the consideration of weights, as $3/5$ is multiplied by 0.5 to account for the weight of `msup`, and 0.5 again to account for the weight of `mfrac`, both of which are tags only present in one equation and not the other.

At this point the depth score is calculated, and the final and complete depth score of the whole tree/equation is calculated by taking the average of all the depth scores with equation (3). The depth score for each branch and the weights fall between 0 and 1, and therefore the depth score for the whole equation also falls within this bound. A score of 0 means the equations have nothing in common and 1 means the opposite.

Table (2): List of all weighted tags and their respective weights.

Tag	Weight
<mover>	0.005
<munder>	0.005
<munderover>	0.005
<mfrac>	0.5
<msup>	0.5
<msubsup>	0.5
<msqrt>	0.5

Table (3): List of all MathML tags.

Tag	Meaning
<math>	Top level element to indicate the start of a single math formula.
<mfrac>	Branches into two children. First is the numerator, second is the denominator.
<mrow>	Groups items together. Functions like parentheses.
<msqrt>	Square root.
<mroot>	Branches into two children. First is in the root, the second is the degree of the root.
<msub>	Branches into two children. First is base, second is subscript.
<msup>	Same as above except second child is superscript.
<msubsup>	Branches into three children. First is base, second is subscript, third is superscript.
<munder>	Branches into two children. First is base, second is under.
<mover>	Branches into two children. First is base, second is over.
<munderover>	Branches into three children. First is base, second is under, third is over.
<mn>	Wraps a single literal (decimal or int).
<mi>	Wraps a single "identifier" (variable, function (cos, sin, etc...), symbol (pi, theta, etc.)).
<mo>	Wraps any single operator (+, -, etc.).
<mtext>	Wraps text (if, maps to, etc.).

Complexity Score

The second part of the final similarity score is the complexity score, calculated with equation (1). This equation yields a number between 0 and 1, 1 representing both trees having the exact same amount of MathML tags, and 0 the opposite.

The complexity score is meant to account for the case where one equation is significantly more complex than the other, which can be seen by comparing the number of tags found in the Left Tree and Right Tree. Since the depth score allows multiple branches in the Right Tree to match the same branch in the Left Tree, relying solely on the depth score can result in an inflated total score that does not accurately reflect the (dis)similarity between the two equations, and that problem is corrected with the inclusion of the complexity score.

For example, given a Right Tree referring to the equation $\frac{2}{3}$ and a Left Tree referring to the equation $A_1 + B_1 * A_2 - \frac{11}{a} * \frac{b}{555}$, the depth score would be 1, meaning that the equations are exactly similar. This is due to the fact that the branch with a constant in the numerator in the Right Tree matches with the branch that has a constant in the numerator in the Left Tree, and same for the denominator. The depth score considers how much the Right Tree has in common with the Left Tree, and in many cases considering the similarities will account for the dissimilarities, but there are instances like above where that is not the case. In an instance like this, the depth score accounts for what the equations have in common, but the complexity score is needed to account for what the equations do not have in common. Considering both the depth score and complexity score yields a more accurate total score than if only the depth score were considered.

The complexity score accounts for structural differences between equations by penalizing cases where one tree is significantly more complex than the other, ensuring the total score is accurate and the potential pitfalls of the depth score are avoided. While this could theoretically be achieved by not only considering how much the Right Tree has in common with the Left Tree but also the inverse (calculating another depth score for how much the Left Tree has in common with the Right Tree), such an approach would significantly increase computational complexity and would not be significantly more accurate than the complexity score to justify that. The complexity score achieves comparable results with greater efficiency and without unnecessary complexity, reinforcing the proposed algorithm's focus on speed and simplicity.

Total Score

The final score is calculated with equation (2), which combines the depth and complexity score in a way that weighs their contributions based on the depth score. As seen in equation (2), the complexity score is weighted depending on the depth score because its purpose is to lower a disproportionately high depth score, not to increase the total score when the depth score is low. Hence why when the depth score is high, the complexity score has a greater impact on the total score, and when the depth score is low, the complexity score has a lesser impact on the total score. In Equation (2), because the complexity score is multiplied by the depth score, the higher the depth score, the more weight the complexity score has on the total score. And due to depth

score being multiplied by 1 - depth score, the higher the depth score, the lower the number depth score is multiplied by, and therefore the lower the weight of the depth score on the total score. Because depth score and complexity score are both bounded by 0 and 1, 1 representing similarity and 0 the opposite, the same applies for the total score.¹

$$\text{complexity_score} = \min(LTT, RTT) / \max(LTT, RTT),$$

where LTT and RTT are the number of MathML tags in the Left Tree and RightTree respectively.

Equation (1)

$$\text{final_score} = \text{complexity_score} * \text{total_depth_score} + \text{total_depth_score} * (1 - \text{total_depth_score})$$

Equation (2)

$$\text{total_depth_score} = \sum_{i=1}^n \frac{\text{depth_score}_i}{n}$$

Equation (3)

Experimental Results

The performance of the proposed algorithm is analyzed using the MSE database [15], which contains over 1.5 million questions with corresponding answers and user ratings for the answer. A higher rating means more users found the answer relevant to their question, and a lower (and sometimes negative) rating indicates the answer was deemed less relevant. The dataset² separates the textual content of questions and answers from the MathML, which allows us to easily input only the equation portion of a question/answer into the proposed algorithm. From the dataset, we selected 10 questions, each with at least five answers. Both the questions and their answers included a number of math equations. These 10 questions are their respective answers make up the test cases that will assess the novelty of the proposed algorithm. The highest rated answer to a question is always considered in each test case. The ability of the algorithm to recommend the most similar math equation is measured against GPT4.0, in order to assess the proposed algorithm's speed and accuracy.

GPT-4.0 [16] is a generative pre-trained transformer designed for advanced natural language processing tasks. It is trained on a diverse and extensive set of data, including web content, books, and other written sources, encompassing trillions of tokens which serves to build its neural network. The model utilizes unsupervised learning to develop a deep understanding of language, enabling it to generate coherent and contextually relevant responses across a wide range of topics. While not specialized for anything specific beyond natural human interaction, its ability to understand context and language make it an effective tool for recommending math answers nonetheless.

¹ The complete code for the proposed algorithm is available at <https://github.com/ElemehnoP/CS497R>

² The complete dataset is available at <https://drive.google.com/drive/folders/1ZHxal8Jyw0TtHwabgS0mZyTmQ9Fwf9P?usp=sharing>

To evaluate the proposed algorithm, it is run using data from the 10 selected questions and the respective answers. For each test case, the algorithm is run five times using the same setup: one equation from the question in the test case was kept fixed, while the other equation was varied to correspond to a different answer for the same question. Each run of the algorithm computes a similarity score between the equation found in the question and the equation found in the answer, and the test returns a list of the equations compared and how similar it determined each pair was. The highest similarity score among each pair is returned as the final score of the question and answer.

GPT-4.0 is tasked with performing the same tests by being provided with files containing the equations and being prompted to assess the similarity of the equations, and to determine how similar each answer equation is to the question equation.

The results of the proposed algorithm and GPT-4.0 are summarized in Table (1). The table presents the average time required for the math recommender to complete each test, as well as its performance in identifying the most similar equation. The accuracy of the recommender is calculated based on whether the equation from the highest-rated answer is deemed the most similar to the equation in the question. For each test, if the highest rated answer's equation is correctly identified as the most similar, the accuracy score is 1; if not, it is 0. These individual accuracy scores are then averaged to produce the overall accuracy, as shown in the table. The columns include "Average Time" for the time taken to complete each test and "Average Accuracy" for the overall accuracy of the recommender system.

Table (1): Speed and accuracy of the tested math recommenders.

	Average time (seconds)	Average accuracy
GPT-4.0	4.296	0.8
Proposed algorithm	0.114	0.8

The results demonstrate that the proposed algorithm outperforms GPT-4.0 in terms of execution time while maintaining accuracy. As indicated by the "Average Time" column in Table (1), the proposed algorithm completes each test significantly faster than GPT-4.0, which can be attributed to its simplicity. Despite this speed advantage, the "Average Accuracy" column shows that the proposed algorithm matches GPT-4.0 in its ability to correctly identify the most similar equation from the set of answers. This suggests that the proposed algorithm, while less computationally intensive, is just as effective as GPT-4.0 in the task of determining mathematical similarity. The results highlight the efficiency of the proposed algorithm, offering a practical alternative to more complex models like GPT-4.0 when it comes to the task of processing the similarity of math equations as a way to better recommend existing math answers to math questions.

Conclusion

Math is a very diverse and important field of study. It is relevant to daily life directly and indirectly, as it is used in manufacturing cars and inventing new medicine, as well as shopping and finance management. Better math skills lead to better decision making capabilities [18], but math learning is inhibited by the fact that there is a general reluctance to engage with it due to its logical thinking and sometimes its complexity. Making math learning more accessible is crucial to solving this problem, and math Q&A platforms address this need. However, the inconvenience of not receiving a timely answer to a question can be a roadblock to math learning. Math Q&A systems attempt to mitigate this issue by recommending existing answers to questions, but its effectiveness is heavily dependent upon the quality of the recommendation. Poorly matched recommendations can discourage users, and so we propose an algorithm that combines the idea of matching using unification with a novel scoring system to attempt to improve the quality of the recommended answers. The proposed algorithm is notable for its unique design, as well as its speed and accuracy. Conducted empirical analysis has proven the novelty and the effectiveness and efficiency of the proposed system.

Figure 1.0: Standard representation of the MathML equation in Figure 1.1.

$$\frac{\sqrt{a^2 - a}}{b} + c^4$$

Figure 2.0: Standard representation of the MathML equation in Figure 2.1.

$$\frac{x^2 + y}{z} - f$$

Figure 1.1: MathML representation³ of the equation in Figure 1.0. The tree representation of this data can be seen in Figure 1.3.

```
<math xmlns="http://www.w3.org/1998/Math/MathML">
  <mrow>
    <mfrac>
      <mrow>
        <msqrt>
          <mrow>
            <msup>
              <mi>a</mi> (8)
              <mn>2</mn> (7)
            </msup>
            <mo>-</mo> (6)
            <mi>a</mi> (5)
          </mrow>
        </msqrt>
      </mrow>
      <mo>+</mo> (5)
    </mfrac>
    <msup>
      <mi>c</mi> (4)
      <mn>4</mn> (6)
    </msup>
  </mrow>
</math>
```

Figure 2.1: MathML representation³ of the equation in Figure 2.0. The tree representation of this data can be seen in Figure 2.3.

```
<math xmlns="http://www.w3.org/1998/Math/MathML">
  <mrow>
    <mfrac>
      <mrow>
        <msup>
          <mi>x</mi> (7)
          <mn>2</mn> (6)
        </msup>
        <mo>+</mo> (5)
        <mi>y</mi> (4)
      </mrow>
      <mi>z</mi> (3)
    </mfrac>
    <mo>-</mo> (5)
    <mi>f</mi> (4)
  </mrow>
</math>
```

³ The portion in parenthesis is meant only to show which array in Figure 1.2/Figure 2.2 corresponds to which branch in the MathML representation; it is not a part of the actual MathML file.

```

<mrow>
  <mi>b</mi> (4)
</mrow>
<mfrac>
<mo>+</mo> (3)
<msup>
  <mi>c</mi> (2)
  <mn>4</mn> (1)
</msup>
</mrow>
</math>

```

```

<mo>-</mo> (2)
<mi>f</mi> (1)
</mrow>
</math>

```

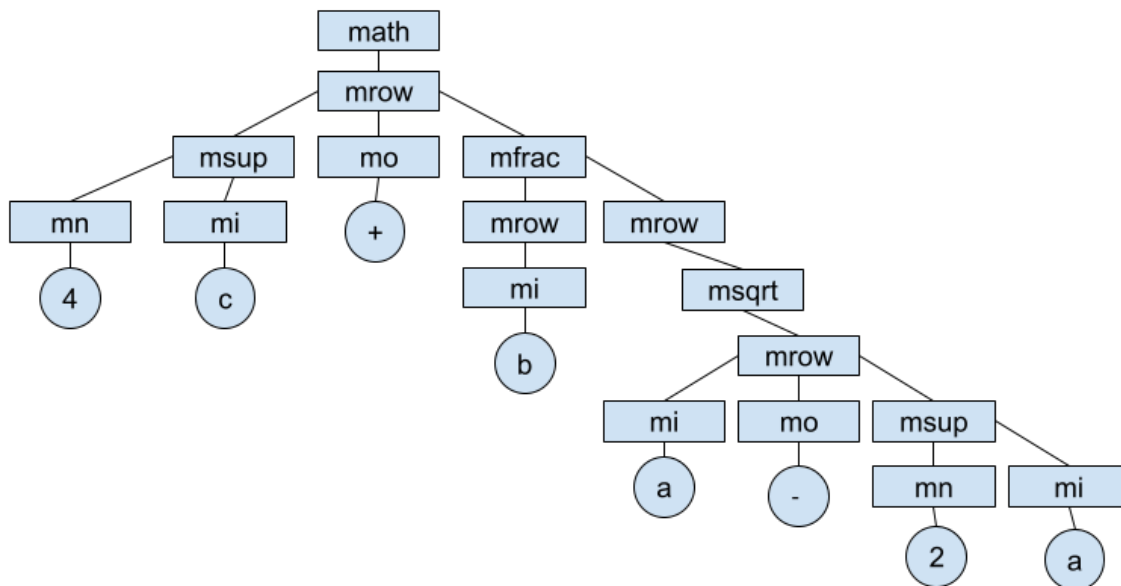
Figure 1.2: The branch array⁴ of the equation found in Figure 1.0 and Figure 1.1.

1. [math, mrow, msup, 1, mn], (4)
2. [math, mrow, msup, 0, mi], (c)
3. [math, mrow, mo], (+)
4. [math, mrow, mfrac, 1, mrow, mi], (b)
5. [math, mrow, mfrac, 0, mrow, msqrt, mrow, mi], (a)
6. [math, mrow, mfrac, 0, mrow, msqrt, mrow, mo], (-)
7. [math, mrow, mfrac, 0, mrow, msqrt, mrow, msup, 1, mn], (2)
8. [math, mrow, mfrac, 0, mrow, msqrt, mrow, msup, 0, mi], (a)

Figure 2.2: The branch array⁴ of the equation found in Figure 2.0 and Figure 2.1.

1. [math, mrow, mi], (f)
2. [math, mrow, mo], (-)
3. [math, mrow, mfrac, 1, mrow, mi], (z)
4. [math, mrow, mfrac, 0, mrow, mi], (y)
5. [math, mrow, mfrac, 0, mrow, mo], (+)
6. [math, mrow, mfrac, 0, mrow, msup, 1, mn], (2)
7. [math, mrow, mfrac, 0, mrow, msup, 0, mi], (x)

Figure 1.3: Hierarchical tree representation of the MathML equation in Figure 1.1.



⁴ The portion in parenthesis is meant only to show what part of the equation the array represents; it is not a part of the data structure itself, and it is not considered in any calculations.

Figure 2.3: Hierarchical tree representation of the MathML equation in Figure 2.1.

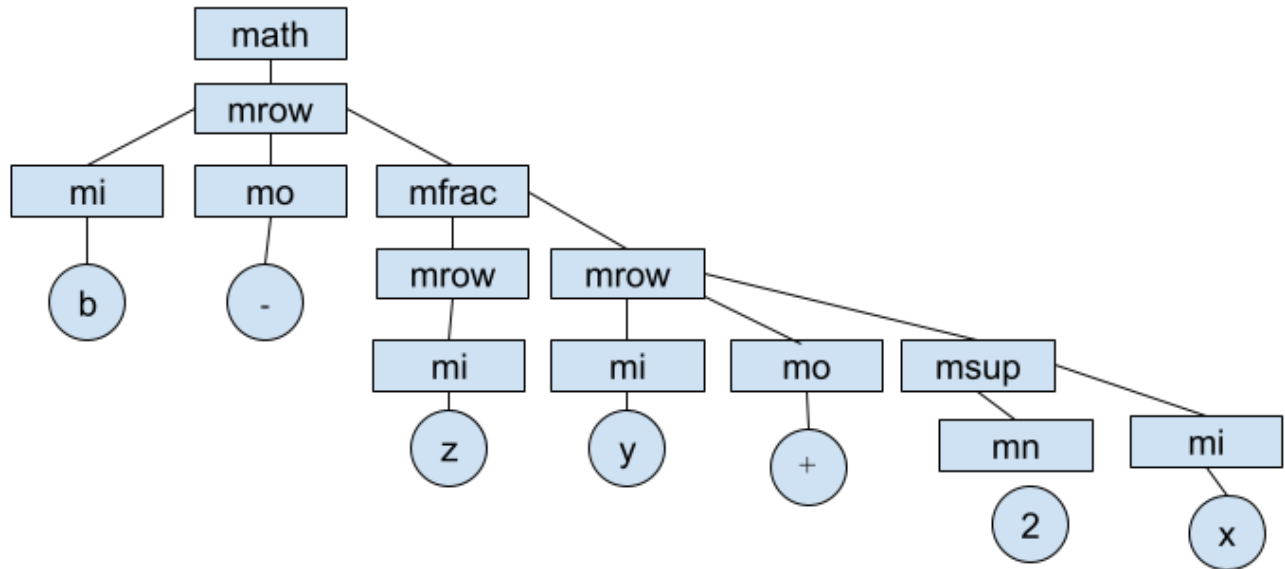


Figure 3.0: Algorithm Pseudocode

```

def compareTrees(Lfile, Rfile) -> float:
    Ltree, Rtree = Python.ElemTree.parse
    LpathsArr = parseTree(Ltree)
    RpathsArr = parseTree(Rtree)
    score: float = scoreTree(LpathsArr, RpathsArr)
    return score
  
```

```

def parseTree(tree):
    pathsArr = [[]]
    deque = []
    while deque is not empty:
        node = deque.pop()
        if node has no children:
            pathsArr.append(path to node)
            continue
        for each childNode of node:
            i = 0
            if childNode in BRANCHING_NODES:
                deque.push(i++)
                deque.push(childNode)
            else:
                deque.push(childNode)
    return pathsArr
  
```

```

BRANCHING_NODES = [
    "mover",
  
```

```

def getBranchInfo(Lpath, Rpath):
    bestDepth = longestContigSubseqCount(Lpath, Rpath)
    score = bestDepth / len(Rpath)
    LbranchingCounts, RbranchingCounts =
        countBranching(Lpath, Rpath)
    weight = getWeight(LbranchingCounts, RbranchingCounts)
    return score, weight
  
```

```

def longestContigSubseqCount(Lpath, Rpath):
    use dynamic programming table to find the size of
    the longest contiguous subsequence found in
    Lpath and Rpath
    return size of longest contiguous subsequence
  
```

```

def countBranching(Lpath, Rpath):
    LbranchingCounts, RbranchingCounts = number of nodes
    in Lpath, Rpath found in BRANCHING_NODES
    return LbranchingCounts, RbranchingCounts
  
```

```

def getWeight(LbranchingNodes, RbranchingNodes):
    MOVER_WEIGHT = 0.005
    MUNDER_WEIGHT = 0.005
    MUNDEROVER_WEIGHT = 0.005
    MFRAC_WEIGHT = 0.5
    MSUP_WEIGHT = 0.5
    MSUBSUP_WEIGHT = 0.5
  
```

```

"munder",
"munderover",
"mfrac",
"msup",
"msubsup",
]
def scoreTree(LpathsArr, RpathsArr) -> float:
    branchScoresArr = []
    for Rpath in RpathsArr:
        for Lpath in LpathsArr:
            branchScore, weight = getBranchInfo(Lpath, Rpath)
            if the branchScore is better than previous branchScore:
                overwrite previous branchScore, weight
            branchScoresArr.append(branchScore * weight)
    depthScore = average values in branchScoresArr
    complexityScore = size of LpathsArr / size of RpathsArr
    depthWeight = 1 - depthScore
    complexityWeight = depthScore
    return complexityScore * complexityWeight +
        depthScore * depthWeight
MSQRT_WEIGHT = 0.5
for each branching node in LbranchingNodes not found in
    RbranchingNodes, increase the weight
    using the above values
return weight

```

References

- [1] Y. K. Wan, C. Hendra, P. N. Pratanwanich, and J. Göke, “Beyond sequencing: machine learning algorithms extract biology hidden in Nanopore signal data,” *Trends in Genetics*, Oct. 2021, doi: <https://doi.org/10.1016/j.tig.2021.09.001>.
- [2] Y. Safadi and J. Haddad, “Optimal combined traffic routing and signal control in simple road networks: an analytical solution,” *Transportmetrica A: Transport Science*, vol. 17, no. 3, pp. 308–339, Jul. 2020, doi: <https://doi.org/10.1080/23249935.2020.1783023>.
- [3] A. Fritz, V. G. Haase, and P. Räsänen, Eds., *International Handbook of Mathematical Learning Difficulties*. Cham: Springer International Publishing, 2019. doi: <https://doi.org/10.1007/978-3-319-97148-3>.
- [4] A. Rattan, C. Good, and C. S. Dweck, “‘It’s Ok — Not Everyone Can Be Good at math’: Instructors with an Entity Theory Comfort (and demotivate) Students,” *Journal of Experimental Social Psychology*, vol. 48, no. 3, pp. 731–737, May 2012, doi: <https://doi.org/10.1016/j.jesp.2011.12.012>.
- [5] “CHARTING A COURSE FOR SUCCESS: AMERICA’S STRATEGY FOR STEM EDUCATION,” 2018. Available: <https://www.energy.gov/sites/prod/files/2019/05/f62/STEM-Education-Strategic-Plan-2018.pdf>
- [6] R. Jiang *et al.*, “How mathematics anxiety affects students’ inflexible perseverance in mathematics problem-solving: Examining the mediating role of cognitive reflection,” *British Journal of Educational Psychology*, vol. 91, no. 1, Jun. 2020, doi: <https://doi.org/10.1111/bjep.12364>.

- [7] T. T. Nguyen, K. Chang, and S. C. Hui, "A math-aware search engine for math question answering system," pp. 724–733, Oct. 2012, doi: <https://doi.org/10.1145/2396761.2396854>.
- [8] S. Gao and Y.-K. D. Ng, "Recommending Answers to Math Questions Based on KL-Divergence and Approximate XML Tree Matching," *Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, pp. 21–31, Nov. 2023, doi: <https://doi.org/10.1145/3624918.3625337>.
- [9] P. Sojka and M. Liška, "Indexing and Searching Mathematics in Digital Libraries," *Lecture notes in computer science*, pp. 228–243, Jan. 2011, doi: https://doi.org/10.1007/978-3-642-22673-1_16.
- [10] Jozef Misutka and L. Galambos, "System Description: EgoMath2 As a Tool for Mathematical Searching on Wikipedia.org," *Lecture Notes in Computer Science*, vol. 6824, pp. 307–309, Jul. 2011, doi: https://doi.org/10.1007/978-3-642-22673-1_30.
- [11] K. Davila and R. Zanibbi, "Layout and Semantics: Combining Representations for Mathematical Formula Search," *the 40th International ACM SIGIR Conference*, pp. 1165–1168, Aug. 2017, doi: <https://doi.org/10.1145/3077136.3080748>.
- [12] Y. Stathopoulos, S. Baker, M. Rei, and S. Teufel, "Variable Typing: Assigning Meaning to Variables in Mathematical Text," *Apollo (University of Cambridge)*, pp. 303–312, Jan. 2018, doi: <https://doi.org/10.18653/v1/n18-1028>.
- [13] R. Zanibbi, D. W. Oard, A. Agarwal, and B. Mansouri, "Overview of ARQMath 2020: CLEF Lab on Answer Retrieval for Questions on Math," *Lecture notes in computer science*, pp. 169–193, Jan. 2020, doi: https://doi.org/10.1007/978-3-030-58219-7_15.
- [14] B. Mansouri, R. Zanibbi, D. W. Oard, and A. Agarwal, "Overview of ARQMath-2 (2021): Second CLEF Lab on Answer Retrieval for Questions on Math," *OpenReview*, 2021. <https://openreview.net/forum?id=1mB2t9eqVs> (accessed Dec. 14, 2024).
- [15] A. Satpute *et al.*, "Can LLMs Master Math? Investigating Large Language Models on Math Stack Exchange," *arXiv.org*, 2024. <https://arxiv.org/abs/2404.00344> (accessed Dec. 20, 2024).
- [16] A. Taloni *et al.*, "Comparative performance of humans versus GPT-4.0 and GPT-3.5 in the self-assessment program of American Academy of Ophthalmology," *Scientific Reports*, vol. 13, no. 1, Oct. 2023, doi: <https://doi.org/10.1038/s41598-023-45837-2>.
- [17] P. Hanrahan, "The Math Behind the Movies," *Frontiers for Young Minds*, vol. 11, Jan. 2024, doi: <https://doi.org/10.3389/frym.2023.1166415>.
- [18] R. Stahnke, S. Schueler, and B. Roesken-Winter, "Teachers' Perception, Interpretation, and Decision-Making: A Systematic Review of Empirical Mathematics Education Research.," *ZDM: The International Journal on Mathematics Education*, vol. 48, pp. 1–27, 2016, Accessed: Dec. 23, 2024. [Online]. Available: <https://eric.ed.gov/?id=EJ1096508>